TATVA / HARDWARE DEPLOYMENT

Why it can't flash a real RISC-V board yet

TATVA compiles an ONNX model into a working RISC-V program. It just can't put that program on a physical board — and the reason is not one missing button.

THE ONE-SENTENCE ANSWER

TATVA doesn't emit generic RISC-V code — it emits code built for **QEMU's virtual machine**, which quietly supplies four things a real board does not: fixed memory addresses, firmware to print through, pre-zeroed memory, and 128 MB of RAM.

THE SIX BLOCKERS

01
The feature was never written
`REAL_HW` exists as a name, but the function behind it just raises an error. There is no flashing tool in the project at all — no OpenOCD, no serial library.
`runner.py:1413`

02
The addresses are QEMU's
Memory is hardcoded to one block at `0x80200000`. Every real board differs, and there is no split between flash and RAM — a distinction real boards depend on.
`runner.py:124`

03
Printing needs absent firmware
Every message goes through firmware called OpenSBI. QEMU boots it for you; a small board has none, so it hangs on the first line it tries to print.
`runner.py:241`

04
Startup skips the real work
Two instructions, then `main`. Real hardware also needs blank memory zeroed, data copied out of flash, and the floating-point unit switched on.
`runner.py:115`

05
The model is far too big
A built BERT-tiny needs ~21 MB of RAM. Most of it is avoidable: weights are written without `const`, so they land in RAM instead of flash.
`runner.py:1062`

06
The timings wouldn't carry
Cycles are converted to milliseconds assuming a 100 MHz clock, written into the code. Real boards run at 144, 160, 320 MHz or more.
`runner.py:1458`

BLOCKER 05, TO SCALE — MEASURED FROM A REAL BUILD

17.5 MB of that is model weights sitting in RAM that never change and belong in flash — one missing `const` keyword. A further 3.4 MB is scratch space fixed at a constant size no matter how small the model is. The program code itself is only 115 KB.

WHAT IT WOULD TAKE

PATH A

A Linux board
VisionFive 2, Milk-V, LicheePi, K230. These already run the firmware TATVA expects and have gigabytes of RAM, so five of the six blockers simply vanish.

Stop building bare-metal: compile a normal Linux program, swap the firmware printing for `printf`, drop the startup code, copy it over SSH and run it. The output format needs no changes.

≈ 1 day to a real silicon measurement

PATH B

A microcontroller board
ESP32-C3, CH32V307, HiFive1. Needs the board abstraction the project is missing:

1. Fix the startup code and mark weights `const` — both are bugs today.
2. Add a `BoardProfile`: addresses, clock, how it prints, how it flashes.
3. Make the memory layout and the printing swappable per board.
4. Size memory from the actual model; refuse to flash when it won't fit.
5. Add `tatva flash` — program the board, read the serial port.

Real work, but it unlocks every board at once

Two of these are bugs on QEMU today: blank memory is never zeroed, and the weights are missing `const`.

Full detail: docs/HARDWARE_GAP.md